



Instituto Infnet

Planejamento de projeto de Ciência de Dados

Projeto de Bloco: Inteligência Artificial Aplicada

Aula 1 · Etapa 1

Prof. Renan Santos

O que vamos construir

Um painel de indicadores sustentáveis do Brasil, com dados abertos do IBGE.

Painel de Indicadores Sustentáveis do Brasil

Dados abertos a serviço da Agenda 2030

Problema: comparar indicadores socioambientais entre as UFs para priorizar ações (ODS 6 e 11).

UFs na amostra

8

Regiões cobertas

5

População somada (2025)

114.708.065

Amostra dos dados que usaremos no projeto

uf	estado	regiao	populacao_2025
AM	Amazonas	Norte	4.321.616
PA	Pará	Norte	8.711.196
BA	Bahia	Nordeste	14.870.907
CE	Ceará	Nordeste	9.268.836
DF	Distrito Federal	Centro-Oeste	2.996.899
RJ	Rio de Janeiro	Sudeste	17.223.547
SP	São Paulo	Sudeste	46.081.801
RS	Rio Grande do Sul	Sul	11.233.263

Fonte: IBGE — População residente estimada (2025), tabela 6579.

De onde você vem — e para onde vamos

O bloco foi desenhado para quem vem direto do bloco de entrada — e constrói a partir daí.

- **Você já domina (ou quase) Python básico**

Variáveis, if/for, funções, listas e dicionários.

É esse repertório que usaremos hoje — sem pré-requisito escondido.

- **Um projeto contínuo, etapa a etapa**

Cada encontro acrescenta uma peça ao MESMO app (não são aulas soltas).

Hoje: planejamento.

- **Para onde vamos**

Ao fim: um dashboard que coleta dados da web, consome APIs e usa LLMs.

Construído por você, incrementalmente.

► **Você não começa do zero — começa do Python que já sabe e sobe um degrau por aula.**

Onde estamos no Projeto de Bloco

10 etapas até um dashboard com WebScraping, APIs e LLMs. Hoje: Etapa 1.

Você está aqui



Como vamos aprender neste bloco

Três princípios que valem o bloco inteiro.

- **Ferramenta nova só quando resolve um problema real**
Ex.: pandas só quando o volume de dados justificar.
Introduzimos com propósito e explicando o porquê — nunca 'porque sim'.
- **Dados em Python puro primeiro**
Listas e dicionários, que você já domina.
Transparência: você enxerga cada passo, sem 'sensação de lacuna'.
- **IA como copiloto**
Use ChatGPT/Copilot para gerar e explicar código.
Regra: entender e saber explicar — a IA acelera, não pensa por você.

► **Menos decoreba de biblioteca, mais compreensão do que o código faz.**

O que faz um projeto de dados dar errado?

Objetivos da aula

Cinco capacidades — todas são a Competência 1 do PB e alimentam o TP1.

- **Formular o problema de negócio**
Pergunta afiada, KPIs, público e ODS — o que decide todo o resto.
- **Explicar CRISP-DM e TDSP**
As duas metodologias que organizam um projeto de dados.
- **Organizar diretórios e artefatos**
Estrutura de pastas + Project Charter + Data Summary Report.
- **Representar dados em Python puro e ler um CSV**
Lista de dicionários + módulo csv (sem pandas).
- **Construir e rodar uma app Streamlit**
Do script à interface web, sem saber HTML.

O entregável: TP1

O trabalho prático que fecha a Competência 1 — construído ao longo das Etapas 1–2.

- **Tema: ESG e a Agenda 2030**

Todo TP do bloco parte de um problema real de sustentabilidade (ODS).

- **Partes 1–4: proposta + planejamento + app demo**

Exatamente o que praticamos hoje e na Aula 2.

- **Avalia a Competência 1**

Identificar o problema, as metas e as fontes de dados de um app data-driven.

► Ao fim da Aula 2, ~90% do TP1 já estará pronto.



1 · O problema vem primeiro

O caso que vamos seguir

Um fio condutor real: acompanharemos a construção do Painel ODS — tropeços incluídos.

- **O pedido (vago) de uma ONG**

'Queríamos um painel de sustentabilidade do Brasil.'

Se programássemos agora, cada um faria uma coisa diferente.

- **A pergunta que destrava**

'Que DECISÃO vocês precisam tomar?' → priorizar recursos entre UFs.

Isso muda tudo — e é a Fase 1 do método.

- **Onde isto vai dar**

Hoje: o painel no ar. Ao fim do bloco: com saneamento real e IA.

Vamos ver o processo ACONTECER, fase a fase.

► **Guarde este caso: cada conceito da aula reaparece nele, com decisões e becos reais.**

Os 4 elementos do problema

Um problema de negócio diz: que decisão apoiar, para quem, e como saber se deu certo.

- **1) Pergunta afiada — específica, relevante, não ambígua**
Ex.: 'quais UFs priorizar em saneamento?' — boa o bastante para dizer QUAL dado buscar.
- **2) Meta / KPI — número + prazo (SMART)**
Ex.: cobrir as 27 UFs em 4 semanas. Distingue output (o painel) de outcome (a decisão).
- **3) Público-alvo — quem USA vs. quem DECIDE**
Ex.: o analista usa; o gestor decide o investimento. 'A sociedade' não é público-alvo.
- **4) ODS atendido — e por quê**
Ex.: ODS 6 (saneamento), porque o painel expõe desigualdades e orienta prioridades.

► Sem estes 4, não há como decidir qual dado coletar — definir o problema é trabalho técnico.

Pergunta afiada + output vs outcome

A pergunta afiada é o leme: ela decide o dado e a visualização.

- **Afiada = pronta para apontar o dado**

Vaga: 'melhorar o meio ambiente'.

Afiada: 'quais UFs estão atrás em saneamento?' — já diz o dado (IBGE).

- **Output ≠ Outcome**

Output = entregar o painel.

Outcome = o gestor priorizar 3 UFs com base nele. Persiga o outcome.

- **É a primeira tarefa do projeto**

Abre a fase Business Understanding (TDSP).

Cinco minutos aqui poupam horas de retrabalho depois.

► Se a pergunta não diz qual dado buscar, ela ainda não está afiada.

Afiando a pergunta

Compare a versão vaga com a afiada — repare como a segunda já aponta o dado.

- **"Ajudar o meio ambiente"**
→ 'Quais UFs estão atrás em saneamento e devem ser priorizadas?'
Aponta o dado (saneamento por UF) e a comparação.
- **"Mostrar dados de educação"**
→ 'Onde a evasão no ensino médio mais cresceu em 3 anos?'
Aponta a variável (evasão), o recorte (região) e o período.

► **Teste rápido: sua pergunta permite dizer QUAL arquivo/endpoint buscar? Se não, afie mais.**

Que TIPO de pergunta você faz?

Toda pergunta de dados cai em um de quatro níveis — e o nível define o que o projeto entrega.

- **Descritiva — o que aconteceu?**

Ex.: 'cobertura de saneamento por UF hoje?' Exige coletar + agregar + visualizar.

- **Diagnóstica — por que aconteceu?**

Ex.: 'por que o Norte está atrás?' Exige cruzar e comparar variáveis.

- **Preditiva — o que vai acontecer?**

Ex.: 'como estará em 2030 no ritmo atual?' Exige modelo/estatística.

- **Prescritiva — o que fazer?**

Ex.: 'onde investir para maior impacto?' Exige otimização/recomendação.

► **Nosso painel começa DESCRITIVO — e descritivo bem feito já apoia decisão. Com LLMs, subimos de nível.**

Exemplo: nosso painel

O problema do Painel ODS, nos 4 elementos.

- **Pergunta (descritiva): quais UFs priorizar em saneamento?**

Aponta o dado (indicadores por UF)...

...e a visualização (comparar UFs lado a lado).

- **KPI: 27 UFs, comparar em ≤ 3 cliques**

Cobertura + usabilidade, ambas mensuráveis.

Dá para dizer objetivamente se deu certo.

- **Público: gestores e ONGs · ODS 6 e 11**

Quem decide investimento em saneamento/infraestrutura.

ODS 6 (água) e 11 (cidades) — conectados ao problema.

► **Repare: a pergunta afiada já 'entrega' o dado e a tela — é isso que buscamos no seu rascunho (A1).**



2 · Organizar com método

Por que seguir um método?

Método é o mapa que mantém uma equipe alinhada num projeto de dados.

- **O time enxerga etapas e artefatos**

Todos sabem em que fase o projeto está...

...e onde ficam o Charter, os dados e o código.

- **Menos retrabalho e menos 'achismo'**

Decisões seguem um fluxo conhecido em vez de improviso.

Evita becos sem saída e economiza tempo.

- **Enxuto ≠ burocracia**

Usamos só os artefatos que agregam (Charter, Data Summary).

George: 'ver o processo geral ajuda a pensar os passos.'

► As respostas do início ('por que projetos falham?') são exatamente o que o método previne.

CRISP-DM — 6 fases (cíclico)

Cross-Industry Standard Process for Data Mining (1996) — ainda a metodologia de projeto de dados mais usada da indústria.



CRISP-DM · 1) Entendimento do Negócio

Pergunta-chave: que decisão de negócio vamos apoiar — e como saber se deu certo?

- **O que se faz**

Traduzir a 'dor' do cliente em objetivos e uma pergunta afiada.
Definir critérios de sucesso (KPIs) mensuráveis.

- **Entregável → Project Charter**

Escopo, objetivos, KPIs, ODS, público e riscos.
É o contrato do projeto (documento vivo).

- **No Painel ODS**

'Quais UFs priorizar em saneamento?' (descritiva).
KPI: 27 UFs, comparar em ≤ 3 cliques; ODS 6 e 11.

► **Armadilha: pular ao dado sem saber a decisão. É aqui que o projeto se ganha ou se perde.**

CRISP-DM · 2) Entendimento dos Dados

Pergunta-chave: os dados existem, têm qualidade e respondem à pergunta?

- **O que se faz**
Coleta inicial + exploração (EDA): formato, volume, cobertura, faltantes.
Avaliar se os dados permitem responder à pergunta afiada.
- **Entregável → Data Summary / dicionário**
Fontes, tipo de dado e objetivo de uso.
Primeiras estatísticas e problemas encontrados.
- **No Painel ODS**
As 27 UFs estão na API do IBGE? O ano é recente?
O dado vem em JSON (semi-estruturado).

▶ **Armadilha: confiar na fonte sem inspecionar — dado ruim contamina todo o resto.**

CRISP-DM · 3) Preparação dos Dados

Pergunta-chave: como transformar o dado bruto no dataset pronto para análise?

- **O que se faz**

Limpar (faltantes, erros), converter tipos, integrar fontes, derivar variáveis.
Costuma ser a MAIOR parte do tempo do projeto.

- **Entregável → dataset de análise**

Tabela limpa e consistente, pronta para uso.
Reprodutível: um script, não cliques manuais.

- **No Painel ODS**

JSON da API → lista de dicionários; converter população para inteiro.
Padronizar nomes de UF/região.

▶ **Armadilha: subestimar esse esforço no cronograma — 'garbage in, garbage out'.**

CRISP-DM · 4) Modelagem

Pergunta-chave: qual técnica responde à pergunta — da mais simples à mais sofisticada?

- **O que se faz**
Escolher e aplicar a técnica: de uma agregação/ranking simples a ML.
Testar alternativas e ajustar parâmetros.
- **Entregável → modelo/análise**
O resultado que responde à pergunta (ex.: ranking de UFs).
Com as premissas documentadas.
- **No Painel ODS**
Nesta etapa, uma agregação simples (comparar/ordenar UFs) já basta.
Modelos preditivos e LLMs vêm mais adiante no bloco.

► **Armadilha: modelo sofisticado sobre pergunta mal posta — comece simples.**

CRISP-DM · 5) Avaliação

Pergunta-chave: o resultado responde ao NEGÓCIO — não só à métrica técnica?

- **O que se faz**

Confrontar o resultado com os KPIs e o objetivo de negócio.
Decidir: seguir para produção (go) ou iterar (no-go).

- **Entregável → decisão go/no-go**

Relatório de avaliação + próximos passos.
Se 'no-go', volta ao Negócio ou aos Dados.

- **No Painel ODS**

O gestor consegue priorizar 3 UFs em ≤ 3 cliques?
Se não, refinamos a pergunta ou a visualização.

► **Armadilha: comemorar acurácia técnica enquanto o KPI de negócio não é atingido.**

CRISP-DM · 6) Implantação

Pergunta-chave: como entregar o resultado a quem vai decidir com ele?

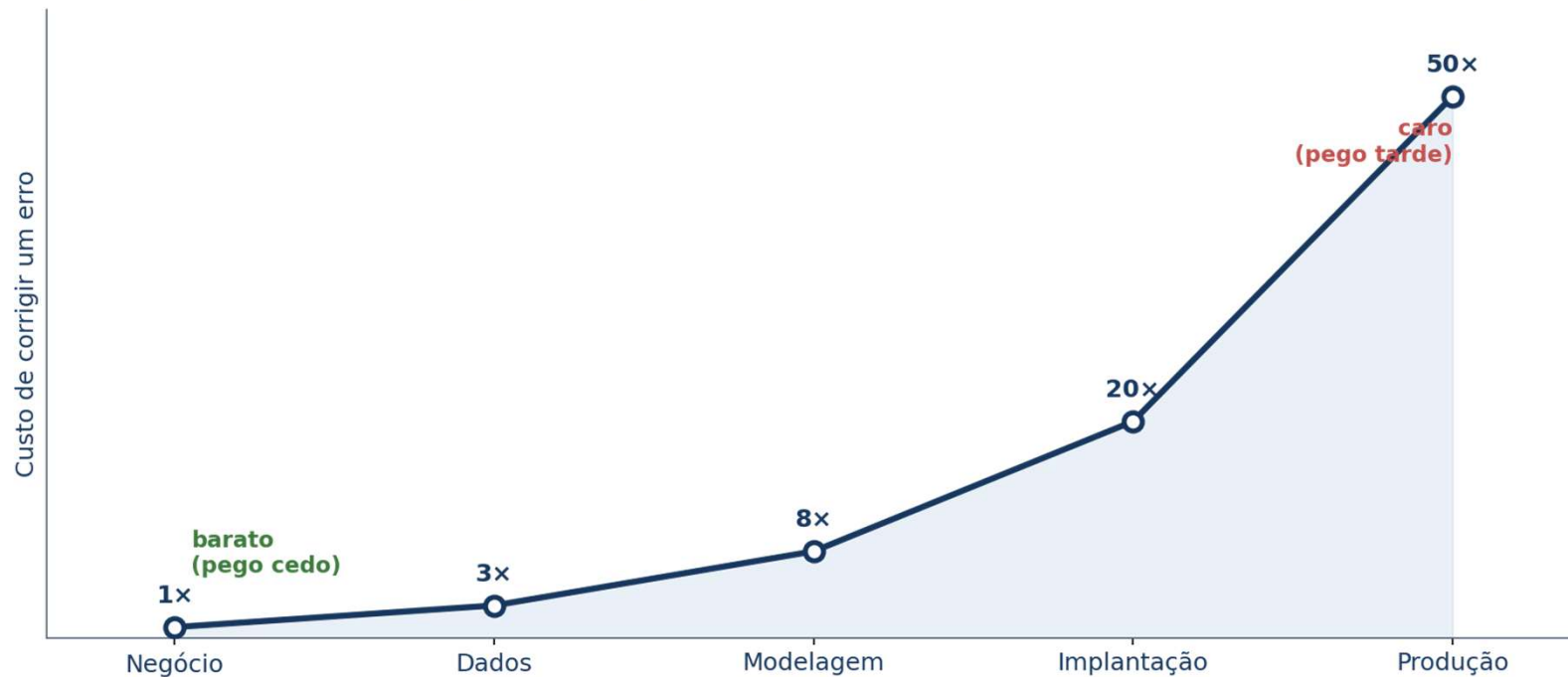
- **O que se faz**
Publicar/operacionalizar (app, relatório, pipeline).
Documentar o uso e monitorar.
- **Entregável → produto em produção**
No nosso caso: o Streamlit acessível.
Com plano de manutenção.
- **No Painel ODS**
Publicar o painel (etapa futura do bloco).
E acompanhar se gestores/ONGs realmente usam.

► **Cíclico: da Avaliação volta-se ao Negócio. Armadilha: entregar e sumir (sem outcome).**

Por que planejar primeiro?

Um erro na PERGUNTA custa minutos se pego na Fase 1 — e o projeto inteiro se pego só em produção.

Curva de custo-da-mudança: erro pego tarde custa muito mais



CRISP-DM: forças e limitações

Por que ainda domina — e o que deixa de fora.

- **Forças**

Universal, simples e independente de ferramenta; a linguagem comum da área desde 1996.
Ótimo para pensar o problema e os dados.

- **Limite 1 — sem papéis nem visão de equipe**

Não diz quem faz o quê; vira problema em times.
(é o primeiro ponto que o TDSP resolve)

- **Limite 2 — vira cascata e para no deploy**

A iteração é 'opcional'; muitos rodam em linha reta.
Não cobre monitorar/operar em produção (MLOps).

► **CRISP-DM é um ótimo mapa conceitual, mas incompleto para equipes e produção — daí TDSP e MLOps.**

Anti-padrões clássicos (e a fase que os previne)

Quase todo fracasso em dados é uma fase pulada.

- **'Solução à procura de problema'**

Começar pelo modelo/ferramenta legal, não pela decisão a apoiar.

Previne: Fase 1 (Entendimento do Negócio).

- **'Boiling the ocean'**

Querer resolver tudo de uma vez; nunca entrega.

Previne: escopo enxuto + iteração.

- **'O modelo que ninguém usou'**

Tecnicamente ótimo, sem deploy nem adoção.

Previne: Fases 5–6 (foco no outcome, não só na métrica).

- **'Lixo entra, lixo sai'**

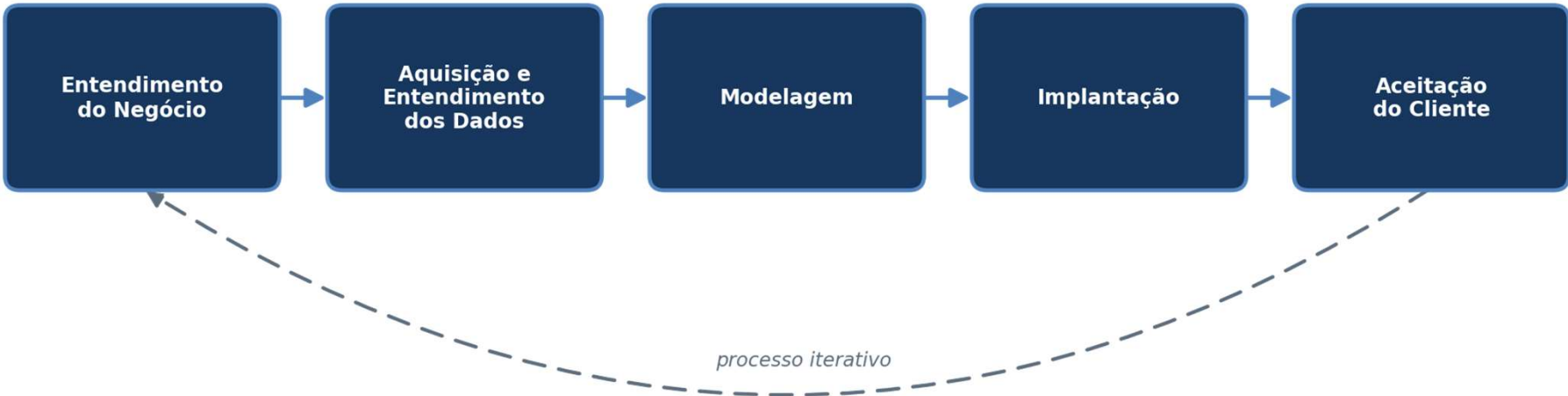
Ignorar a qualidade do dado.

Previne: Fases 2–3 (Entendimento/Preparação).

► O método existe para não pular fases — cada anti-padrão é uma fase negligenciada.

Team Data Science Process (Microsoft, 2016) — evolução moderna do CRISP-DM, pensada para equipes.

TDSP — Team Data Science Process (5 estágios)



TDSP — estágio a estágio

O ciclo de vida do TDSP, com o que acontece em cada estágio.

- **1) Business Understanding**
Define objetivos + perguntas afiadas e JÁ identifica as fontes de dados.
- **2) Data Acquisition & Understanding**
Obtém e explora os dados; monta o pipeline inicial e o dicionário de dados.
- **3) Modeling**
Feature engineering + treino/seleção de modelos que respondem à pergunta.
- **4) Deployment · 5) Customer Acceptance**
Operacionaliza em produção e valida com quem vai usar.

► **Iterativo: cada volta entrega algo utilizável — não se espera o 'fim' para mostrar valor.**

CRISP-DM ↔ TDSP: o mesmo esqueleto

TDSP e CRISP-DM não competem: o TDSP reorganiza o mesmo esqueleto.

- **Entendimento do Negócio = Business Understanding**
Mesma ideia: definir o problema e o sucesso antes de tudo.
- **Entend. + Prep. dos Dados = Data Acquisition & Understanding**
O TDSP funde 'entender' e 'preparar' os dados num estágio só.
- **Modelagem = Modeling**
Correspondência direta entre os dois.
- **Avaliação + Implantação = Deployment + Customer Acceptance**
O TDSP separa 'entregar em produção' de 'ser aceito pelo cliente'.

► Conhecer os dois é o esperado — e o TDSP é a escolha moderna para equipes.

O que o TDSP acrescenta

Por que preferimos o TDSP no Projeto de Bloco.

- **Papéis de equipe**

Líder de projeto, cientista(s) e engenheiro(s) de dados.
Responsabilidades explícitas, sem ambiguidade.

- **Template de repositório (GitHub)**

Estrutura e documentação padronizadas.
O time todo encontra as coisas no mesmo lugar.

- **Iteração com entregas incrementais**

Cada ciclo entrega algo utilizável.
É assim que o painel evolui etapa a etapa neste curso.

► **TDSP = CRISP-DM + papéis + ferramentas + iteração. Por isso é o nosso guia.**

TDSP, Agile e MLOps: como se encaixam

Não são rivais — cada um cobre uma camada diferente.

- **CRISP-DM / TDSP — O QUE fazer**

As fases de um projeto de dados (TDSP = CRISP + papéis + repositório + iteração).

- **Agile / Scrum / Kanban — COMO organizar o trabalho**

Sprints, quadros, entregas curtas. Cientistas de dados adotaram da eng. de software [George, p48].

- **MLOps — OPERAR em produção**

Deploy contínuo, monitoramento, retreino. É a 'Fase 6+' que o CRISP-DM não cobria.

► **Combine: TDSP dá as fases, Agile organiza o trabalho, MLOps mantém em produção.**

Por que 'iterativo' importa

Um exemplo concreto de por que voltar atrás é parte do processo.

- **Modelagem pronta → Avaliação: o KPI NÃO bate**
Ex.: o painel compara UFs, mas o gestor ainda não decide com ele.
O projeto não 'falhou' — aprendeu algo.
- **Não jogamos fora — voltamos atrás**
Ao Negócio (refinar a pergunta) ou aos Dados (nova fonte/indicador).
Cada volta corrige o rumo.
- **Cada volta é uma entrega incremental**
Aula 1 (amostra) → Aula 2 (API) → final (com IA).
Sempre há algo funcionando para mostrar.

► Iterar é o oposto de 'tudo ou nada': entrega-se valor cedo e melhora-se a cada ciclo.

Nosso projeto no TDSP

Onde estamos, na prática, no ciclo do TDSP.

- **Business Understanding = problema + Project Charter**
Pergunta afiada (saneamento por UF), KPIs, ODS 6/11 e público-alvo.
- **Data Acquisition = IBGE + Data Summary**
Fonte: API do IBGE (JSON); registramos tipo e objetivo de uso de cada uma.
- **Modeling / Deployment / Acceptance**
Indicadores derivados e LLMs; publicar o app; validar — etapas futuras.

► Hoje cobrimos o 1º estágio inteiro e começamos o 2º — exatamente as Partes 1–4 do TP1.



3 · Estrutura, tipos de dados e Python puro

Diretórios que espelham o TDSP

painel_ods_brasil/

painel_ods_brasil/

- |— app.py # aplicação Streamlit
- |— requirements.txt # dependências (na RAIZ) – só streamlit
- |— data/ (raw / processed / sample)
- |— docs/ (project_charter.md, data_summary_report.md)
- |— src/ (data_access.py – coleta via API, na Aula 2)

Artefato 1 — Project Charter

O Project Charter responde: o que faremos, para quem e como medir sucesso.

- **Escopo, objetivos e KPIs**

O que está dentro/fora; o que buscamos.
Sucesso com número e prazo (SMART).

- **ODS e público-alvo**

A qual objetivo da Agenda 2030 contribui e para quem.
Ancora todas as decisões do projeto.

- **Riscos — e é documento vivo**

Ex.: 'API fora do ar' → mitigação com cache de amostra.
Revisado ao fim de cada etapa.

▶ **É o artefato da fase Business Understanding — o primeiro que você entrega no TP1.**

Artefato 2 — Data Summary Report

O Data Summary Report cataloga de onde virão os dados — antes de coletar.

- **Fontes de dados**

Ex.: API do IBGE (população; depois saneamento).
Cada fonte listada explicitamente.

- **Tipo de dado**

Estruturado / semi / não-estruturado.
Define como trataremos cada fonte.

- **Objetivo de uso**

Para que cada fonte serve no painel.
Evita coletar dado que ninguém vai usar.

► **Primeiro esboço da fase de Aquisição — a ponte para a coleta real da Aula 2.**

Que TIPO de dado é cada fonte?

Antes de coletar, classificamos o dado — porque o tipo muda como o tratamos.

- **Estruturado — tabelas / CSV / SQL**

Linhas e colunas; ótimo para análise quantitativa. Ex.: nosso CSV de população.

- **Semi-estruturado — JSON, XML**

Tem estrutura, mas não tabular; típico de APIs. 'JSON is a semi-structured format' (Esppenchutz).

- **Não-estruturado — texto, áudio, vídeo, imagem**

Sem esquema fixo; guardado em bancos NoSQL (ex.: MongoDB) e data lakes.

- **Fio do projeto: JSON (semi) → tabela (estruturado)**

A API do IBGE devolve JSON; nós o convertemos numa tabela para comparar UFs.

► Registramos o tipo de cada fonte no Data Summary Report — decide o tratamento na Aula 2.

Bastidores: 3 tropeços reais

Este painel foi construído de verdade — e o dado real derrubou as primeiras tentativas.

- **Saneamento era complexo demais (por ora)**

As tabelas do Censo têm múltiplas classificações — difíceis de extrair limpo na v1.
Decisão: começar por POPULAÇÃO (dado limpo) e adicionar saneamento na iteração.

- **A API respondeu comprimida (gzip)**

A 1ª tentativa com urllib quebrou ao decodificar.
Solução: usar requests, que descomprime sozinho. Sempre inspecione a resposta!

- **KeyError: os ids vinham como texto**

Buscar a UF por número (13) falhou — o id era a string '13'.
Solução: normalizar os tipos (int). O dado real raramente vem no tipo esperado.

► É por isso que Entendimento e Preparação dos Dados existem — o dado real surpreende.

Como guardamos os dados? (Python puro)

No código, uma tabela é só uma lista de dicionários — Python que você já conhece.

- **Começamos com o que você domina**

Listas e dicionários.

Nada de instalar biblioteca nova agora.

- **Cada LINHA = um dicionário {coluna: valor}**

Ex.: {'uf': 'SP', 'estado': 'São Paulo', 'populacao_2025': 46081801}.

As chaves são as colunas.

- **A TABELA = uma lista de dicionários**

Você percorre e filtra com for/list comprehension.

Transparente: cada registro é visível.

▶ É a representação que o Streamlit aceita direto — por isso não precisamos de pandas hoje.

IA como copiloto

A coordenação encoraja usar IA para programar — com uma regra clara.

- **Peça à IA para gerar e explicar trechos**

Ex.: 'como somar uma coluna de uma lista de dicionários?'

Ótimo para destravar rápido.

- **Para aprender — não para terceirizar o pensar**

A IA acelera.

A compreensão continua sendo sua responsabilidade.

- **Regra de ouro: entender e saber explicar**

Se não sabe explicar uma linha, pergunte 'por quê?' à IA.

Só entregue quando entender.

► Quem entende o que a IA gerou usa a ferramenta; quem não entende é usado por ela.

O caso, em 4 lições

O que o Painel ODS nos ensinou — e que vale para o seu TP1.

- **Afiar a pergunta (e o tipo) evita construir a coisa errada**
Descritiva, não preditiva — a ONG precisava decidir agora.
- **Começar simples e descritivo entrega valor cedo**
População + amostra, em vez de travar no dado difícil.
- **Sempre inspecionar a fonte real**
gzip e tipos derrubaram as 1^{as} tentativas — por isso Entendimento/Preparação existem.
- **Iterar: a v1 é degrau para a v2**
População/8 UFs/local → saneamento/27 UFs/API (a Aula 2).

► **Método não é burocracia: foi ele que nos fez começar certo e não desperdiçar tempo.**

Em 3 frases

O mapa mental da Etapa 1.

- **Problema primeiro**
Pergunta afiada + tipo, KPI, público, ODS — antes do código.
- **Método + dados**
CRISP-DM ↔ TDSP (iterativo) + artefatos; tipos de dados; Python puro.
- **App + IA copiloto**
Streamlit põe o app no ar; a IA acelera (entender e explicar).

► **Planejar antes de programar não atrasa — é o que faz o projeto chegar ao fim.**

Para a Aula 2, traga

- Problema nos 4 elementos + tipo da pergunta
- Repositório com a estrutura de pastas
- Rascunho do Project Charter